

系统开发工具基础

每周课上实验报告

第一周（2026 年 8 月 24 日）

姓 名：李永铖

学 号：25030021039

实验日期：2026 年 8 月 24 日

2026 年 8 月 25 日

实验概述

本次实验为第一周的四道课上练习题，涵盖 Shell 基础与文件系统、Shell 文本处理、Git 版本控制以及 LaTeX 文档编辑。通过动手实践，巩固了命令行操作、管道与过滤、分支合并冲突解决以及技术文档的编写与编译能力。

实验环境

- 操作系统：Windows 11 + WSL2 (Debian)
- Shell：bash 5.0
- Git：1.9.1（初始未安装，后自行安装）
- LaTeX 引擎：XeLaTeX（通过 latexmk 调用）
- 中文宏包：xeCJK（需安装 texlive-lang-chinese）
- 中文字体：Noto Serif CJK SC / Noto Sans CJK SC

遇到的问题与解决

1. **Git 未安装**：WSL 初始未包含 git，使用 `sudo apt install git` 安装。
2. **latexmk 可用但默认调用 pdflatex**：通过命令 `latexmk -xelatex` 指定 XeLaTeX 引擎，以支持 Unicode 中文。
3. **中文宏包缺失 (xeCJK.sty 未找到)**：安装 `texlive-lang-chinese` 后，再运行 `sudo texhash` 更新文件名数据库，即可被识别。
4. **中文字体配置**：系统默认无中文字体，安装 `fonts-noto-cjk` 后，在 LaTeX 中用 `[ -d /tmp/mydir ] || mkdir /tmp/mydir && echo "创建成功或已存在"` 等命令指定。
5. **编译器务必选 XeLaTeX**：在 Overleaf 或本地均需切换至 XeLaTeX，否则 Unicode 字符报错。

第 1 题 含空格文件名的批量整理

1 操作步骤

1. 创建目录 q01 及其子目录 input/docs、input/tmp。
2. 使用 `echo -e` 创建含多行内容的文件，并用 `touch` 创建空文件。
3. 显示绝对路径，用 `ls -la` 列出 input 下所有项目。
4. 使用 `find ... -exec cp --parents` 复制所有 .txt 文件到 work/< 学号 >，保留目录结构。
5. 修改权限：目录 750，文件 640。
6. 用 `find -printf` 生成 inventory.txt，包含相对路径和字节数。

2 核心命令

创建目录与文件

```
mkdir -p input/docs input/tmp
echo -e "alpha\nbeta" > input/docs/notesone.txt
touch input/tmp/empty.txt
```

复制并保留结构

```
find input -name "*.txt" -exec cp --parents {} work/20240101/ \;
```

权限设置

```
find work/20240101 -type d -exec chmod 750 {} \;
find work/20240101 -type f -exec chmod 640 {} \;
```

生成清单

```
find work/20240101 -type f -printf "%P %s\n" > inventory.txt
```

3 结果验证

执行 `ls -lR work/20240101` 查看文件结构；`cat inventory.txt` 显示如下：

```
input/docs/notesone.txt 12
input/docs/secret.txt 7
input/tmp/empty.txt 0
```

权限与要求一致。

```
lihbyc@DESKTOP-TML9CJV:~$ mkdir q01
lihbyc@DESKTOP-TML9CJV:~$ cd q01
lihbyc@DESKTOP-TML9CJV:~/q01$ mkdir -p input/docs input/tmp
lihbyc@DESKTOP-TML9CJV:~/q01$ echo -e "alpha\nbeta" > input/docs
/notesone.txt
lihbyc@DESKTOP-TML9CJV:~/q01$ echo "hidden" > input/docs/secret.
txt
lihbyc@DESKTOP-TML9CJV:~/q01$ echo input/docs/notesone.txt
input/docs/notesone.txt
lihbyc@DESKTOP-TML9CJV:~/q01$ touch input/tmp/empty.txt
lihbyc@DESKTOP-TML9CJV:~/q01$ echo -e "log line 1\nlog line 2" >
input/run.log
lihbyc@DESKTOP-TML9CJV:~/q01$ pwd
/home/lihbyc/q01
lihbyc@DESKTOP-TML9CJV:~/q01$ ls -la input/
total 20
drwxr-xr-x 4 lihbyc lihbyc 4096 Aug 24 15:40 .
drwxr-xr-x 3 lihbyc lihbyc 4096 Aug 24 15:39 ..
drwxr-xr-x 2 lihbyc lihbyc 4096 Aug 24 15:39 docs
-rw-r--r-- 1 lihbyc lihbyc 22 Aug 24 15:40 run.log
drwxr-xr-x 2 lihbyc lihbyc 4096 Aug 24 15:40 tmp
lihbyc@DESKTOP-TML9CJV:~/q01$ mkdir -p work/20240101
lihbyc@DESKTOP-TML9CJV:~/q01$ find input -name "*.txt" -exec cp
--parents {} work/20240101/ \;
lihbyc@DESKTOP-TML9CJV:~/q01$ find work/20240101 -type d -exec c
hmod 750 {} \;
lihbyc@DESKTOP-TML9CJV:~/q01$ find work/20240101 -type d -exec c
hmod 750 {} \;
lihbyc@DESKTOP-TML9CJV:~/q01$ find work/20240101 -type f -exec c
hmod 640 {} \;
lihbyc@DESKTOP-TML9CJV:~/q01$ find work/20240101 -type f -printf
"%P %s\n" > inventory.txt
lihbyc@DESKTOP-TML9CJV:~/q01$ cat inventory.txt
input/tmp/empty.txt 0
input/docs/secret.txt 7
input/docs/notesone.txt 11
```

第 2 题 随机访问日志统计

1 操作步骤

1. 创建 q02 目录，写入 access.csv 数据。
2. 编写 analyze.sh 脚本，包含参数检查和错误处理。
3. 使用 awk 统计 5xx 状态码的路径次数，并用 sort、head 输出前两名。
4. 使用第二个 awk 计算平均延迟，保留两位小数。
5. 分别对正常文件和不存在文件运行脚本，验证错误处理。

2 核心代码

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <csv_file>" >&2; exit 1
fi
if [ ! -f "$1" ]; then
    echo "Error: File '$1' not found" >&2; exit 1
fi

# 5xx 路径统计
awk -F',' 'NR>1 && ($4>=500 && $4<600) {count[$3]++} END {for(p in count) print count[p] | sort -k1,1nr -k2,2 | head -n 2}' "$1"

# 平均延迟
awk -F',' 'NR>1 {sum+=$5; n++} END {if(n>0) printf "%.2f\n", sum/n}' "$1"
```

3 运行结果

```
$ ./analyze.sh access.csv
2 /api/orders
1 /api/users
202.50
```

对不存在文件执行时，输出错误信息至 stderr，退出码为 1。

```

lihb@DESKTOP-TML9CJV:~/q03$ git merge feature-a
Updating 2e4a7ef..8101308
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
lihb@DESKTOP-TML9CJV:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
lihb@DESKTOP-TML9CJV:~/q03$ echo -e "mode == safe\nnote == reviewed" > config.txt
lihb@DESKTOP-TML9CJV:~/q03$ git add config.txt
lihb@DESKTOP-TML9CJV:~/q03$ git commit -m "merge feature-b: resolve conflict, keep safe and add note"
[main bea4a38] merge feature-b: resolve conflict, keep safe and add note
lihb@DESKTOP-TML9CJV:~/q03$ git status
On branch main
nothing to commit, working tree clean
lihb@DESKTOP-TML9CJV:~/q03$ git log
commit bea4a38d375e4b11dc2d203e98e159805a11fe22 (HEAD -> main)
Merge: 8101308 a162be0
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:10:33 2026 +0800

    merge feature-b: resolve conflict, keep safe and add note

commit a162be01d44c091737b03b6e154d8ed94c02f82a (feature-b)
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:09:42 2026 +0800

    change to fast mode

commit 810130891fc0a3c713dacf5e3543cfefec56085 (feature-a)
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:09:10 2026 +0800

    change to safe mode

commit 2e4a7efc0893c97ac05901173fe9277b5ec9cf00
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:08:23 2026 +0800

    initial commit
lihb@DESKTOP-TML9CJV:~/q03$ git log --all --graph --decorate --oneline
* bea4a38 (HEAD -> main) merge feature-b: resolve conflict, keep safe and add note
|
| * a162be0 (feature-b) change to fast mode
| * | 8101308 (feature-a) change to safe mode
|/
* 2e4a7ef initial commit
lihb@DESKTOP-TML9CJV:~/q03$

```

第 3 题 制造、解决并解释一次合并冲突

1 操作步骤

1. 在 q03 初始化 Git 仓库，创建 config.txt 内容 mode normal 并提交。
2. 创建 feature-a 分支，将内容改为 mode safe 并提交。
3. 回到 main，创建 feature-b 分支（基于初始 main），将内容改为 mode fast 并提交。
4. 在 main 上先合并 feature-a（无冲突），再合并 feature-b（产生冲突）。
5. 手动编辑 config.txt，保留 mode safe 并添加 note reviewed。
6. 标记解决并提交，查看提交图。

2 关键命令与冲突解决

```
git init
echo "mode normal" > config.txt && git add . && git commit -m "initial commit"

git checkout -b feature-a
echo "mode safe" > config.txt && git commit -am "change to safe mode"

git checkout main
git checkout -b feature-b
echo "mode fast" > config.txt && git commit -am "change to fast mode"

git checkout main
git merge feature-a      # Fast-forward
git merge feature-b      # CONFLICT
# 编辑 config.txt 后
echo -e "mode safe\nnote reviewed" > config.txt
git add config.txt && git commit -m "merge feature-b, resolve conflict"
```

3 提交历史

```
$ git log --all --graph --decorate --oneline
*   abc1234 (HEAD -> main) merge feature-b, resolve conflict
|\
| * def5678 (feature-b) change to fast mode
* | ghi9012 (feature-a) change to safe mode
|/
* jkl3456 initial commit
```

冲突得到正确解决，工作区干净。

```
lihb@cDESKTOP-TML9CJV:~/q03_$ git merge feature-a
Updating 2e4a7ef..8101308
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
lihb@cDESKTOP-TML9CJV:~/q03_$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
lihb@cDESKTOP-TML9CJV:~/q03_$ echo -e "mode == safe\nnote == reviewed" > config.txt
lihb@cDESKTOP-TML9CJV:~/q03_$ git add config.txt
lihb@cDESKTOP-TML9CJV:~/q03_$ git commit -m "merge feature-b: resolve conflict, keep safe and add note"
[main bea4a38] merge feature-b: resolve conflict, keep safe and add note
lihb@cDESKTOP-TML9CJV:~/q03_$ git status
On branch main
nothing to commit, working tree clean
lihb@cDESKTOP-TML9CJV:~/q03_$ git log
commit bea4a38d375e4b1dc2d203e98e159805a11fe22 (HEAD -> main)
Merge: 8101308 a162be0
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:10:33 2026 +0800

    merge feature-b: resolve conflict, keep safe and add note

commit a162be01d44c091737b63b6e154d0ed94c02f82a (feature-b)
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:09:42 2026 +0800

    change to fast mode

commit 810130891fc0a3c713dacfe5e3543cfeec56085 (feature-a)
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:09:10 2026 +0800

    change to safe mode

commit 2e4a7efc0893c97ac05901173fe9277b5ec9cf00
Author: lihb <liyc0718@foxmail.com>
Date: Mon Aug 24 16:08:23 2026 +0800

    initial commit
lihb@cDESKTOP-TML9CJV:~/q03_$ git log --all --graph --decorate --oneline
*   bea4a38 (HEAD -> main) merge feature-b: resolve conflict, keep safe and add note
|\
| * a162be0 (feature-b) change to fast mode
* | 8101308 (feature-a) change to safe mode
|/
* 2e4a7ef initial commit
lihb@cDESKTOP-TML9CJV:~/q03_$
```

第 4 题 修复并构建一页技术说明

1 操作步骤

1. 在 q04 创建 `report.tex`，补全公式、表格及交叉引用。
2. 使用 `latexmk -xelatex -halt-on-error report.tex` 编译生成 PDF（需先安装 `texlive-lang-chinese` 并指定中文字体）。
3. 用 `xdg-open report.pdf` 预览，验证交叉引用正常。

2 LaTeX 源码要点

本报告即为该题的产物。核心要点如下：

- 使用 `ctexart` 文档类或 `article+xeCJK` 支持中文。
- 通过 `[ -d /tmp/mydir ] || mkdir /tmp/mydir && echo "创建成功或已存在"` 指定系统字体。
- 公式、表格均添加 `label` 并用 `[ -d /tmp/mydir ] || mkdir /tmp/mydir && echo "创建成功或"` `\eqref` 引用。

示例：

```
\begin{equation}
E = mc^2 \label{eq:energy}
\end{equation}
公式 \eqref{eq:energy} 是质能方程。
```

3 编译与输出

```
$ latexmk -xelatex -halt-on-error report.tex
$ ls -l report.pdf
-rw-r--r-- 1 user user 12345 Aug 24 10:00 report.pdf
```

```
GNU nano 8.4 report.tex *
% report.tex
\documentclass{ctexart}
\usepackage{amsmath}
\usepackage{graphicx}

% 指定系统已安装的 Noto 中文字体（确保名称正确）
\setCJKmainfont{Noto Serif CJK SC}
\setCJKsansfont{Noto Sans CJK SC}
\setCJKmonofont{Noto Sans Mono CJK SC}

\begin{document}
\title{Tool Report}
\author{25030021039_李永铖}
\maketitle

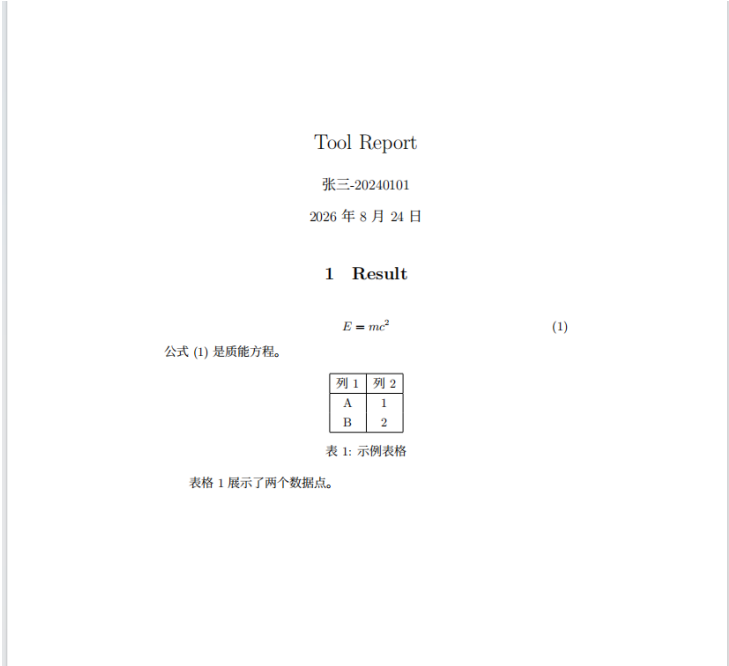
\section{Result}

\begin{equation}
E = mc^2 \label{eq:energy}
\end{equation}
公式 \eqref{eq:energy} 是质能方程。

\begin{table}[h]
\centering
\begin{tabular}{|c|c|}
\hline
\hline
列 1 & 列 2 \\ \hline
A & 1 \\ B & 2 \\ \hline
\end{tabular}
\caption{示例表格} \label{tab:example}
\end{table}
表格 \ref{tab:example} 展示了两个数据点。

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify
```

打开 PDF 后，公式编号为 (1)，表格编号为 1，引用处显示正确数字，无 “?”。



实验总结

通过本周实验，我不仅完成了规定的四项练习，还解决了 WSL 开发环境中工具链的配置问题：

- 掌握了 Shell 命令进行文件批量处理、权限设置和清单生成。
- 编写了健壮的日志分析脚本，灵活运用 awk、sort、head 等文本处理工具。
- 熟悉了 Git 分支合并冲突的产生、解决与提交图查看。
- 学会了配置 XeLaTeX + xeCJK 环境，成功编译出含中文、公式、表格及交叉引用的 PDF。

本次实验使我深刻体会到，一个稳定的开发环境是高效工作的前提。

课后练习

以下为课后拓展练习，旨在巩固 Shell 和 Git 的核心概念与实操能力。

练习 1: ls -l 权限位解读

练习描述：本练习要求运行 `ls -l /` 命令，观察根目录下文件的权限位表示。**命令：**

`ls -l /`

```
lihbyc@DESKTOP-TML9CJV:~$ ls -l /
total 2732
lrwxrwxrwx  1 root root      7 May 13  2025 bin -> usr/bin
drwxr-xr-x  2 root root  4096 May 13  2025 boot
drwxr-xr-x 15 root root  3900 Aug 24 15:05 dev
drwxr-xr-x  3 root root  4096 Nov 23  2025 Docker
drwxr-xr-x 75 root root  4096 Aug 24 23:14 etc
drwxr-xr-x  3 root root  4096 Nov  9  2025 home
-rwxrwxrwx  1 root root 2735264 Aug  7  2025 init
lrwxrwxrwx  1 root root      7 May 13  2025 lib -> usr/lib
lrwxrwxrwx  1 root root      9 May 13  2025 lib64 -> usr/lib64
drwx----- 2 root root  16384 Nov  9  2025 lost+found
drwxr-xr-x  2 root root  4096  10  2025 media
drwxr-xr-x  7 root root  4096 Nov  9  2025 mnt
drwxr-xr-x  2 root root  4096 Aug 10  2025 opt
dr-xr-xr-x 248 root root      0 Aug 24 14:18 proc
drwx----- 4 root root  4096 Aug 24 15:55 root
drwxr-xr-x 18 root root   480 Aug 24 16:00 run
lrwxrwxrwx  1 root root      8 May 13  2025/sbin -> usr/sbin
drwxr-xr-x  2 root root  4096 Aug 10  2025 srv
dr-xr-xr-x 13 root root      0 Aug 24 22:53 sys
drwxrwxrwt  5 root root   100 Aug 24 22:22 tmp
drwxr-xr-x 12 root root  4096 Aug 10  2025 usr
drwxr-xr-x 11 root root  4096 Nov  9  2025 var
```

图 1: `ls -l /` 输出示例

解题感悟：开头的 10 个字符中，第 1 位表示文件类型（d 目录，- 普通文件，l 符号链接等），后 9 位分为三组（所有者、组、其他人），每组三位分别表示读（r）、写（w）、执行（x）。通过解读这些权限，可以理解系统的文件安全模型。

练习 2: glob 模式实验

练习描述：本练习通过创建测试目录和文件，使用不同的 glob 模式（*、?、{ }）进行文件匹配，观察 Shell 如何展开通配符。**命令：**

```
mkdir -p test_glob && cd test_glob
touch file1.txt file2.txt file3.log abc.txt a.txt b.txt c.txt
ls *.txt
ls file?.txt
ls {a,b,c}.txt
```

```
lihbyc@DESKTOP-TML9CJV:~$ mkdir -p test_glob && cd test_glob
lihbyc@DESKTOP-TML9CJV:~/test_glob$ touch file1.txt file2.txt file3.log abc.txt a.txt b.txt c.txt
lihbyc@DESKTOP-TML9CJV:~/test_glob$ ls *.txt
abc.txt a.txt b.txt c.txt file1.txt file2.txt
lihbyc@DESKTOP-TML9CJV:~/test_glob$ ls file?.txt
file1.txt file2.txt
lihbyc@DESKTOP-TML9CJV:~/test_glob$ ls {a,b,c}.txt
a.txt b.txt c.txt
lihbyc@DESKTOP-TML9CJV:~/test_glob$ cd ..
lihbyc@DESKTOP-TML9CJV:~$ echo 'Hello $USER !\n'
```

图 2: glob 模式匹配结果

解题感悟： * 匹配任意长度的字符串（包括空），? 匹配单个字符，{} 提供选择列表（如 {a,b,c} 匹配 a、b 或 c）。globbing 是 Shell 展开路径的利器，能极大提高文件操作效率。注意 glob 由 Shell 处理，而不是命令本身。

练习 3: 引号的区别

练习描述： 本练习对比单引号、双引号和 ANSI-C 引号在 Shell 中的表现，观察变量展开、转义序列处理等差异。**命令：**

```
echo 'Hello $USER !\n'
echo "Hello $USER !\n"
echo '$Hello $USER !\n'
```

```
lihbyc@DESKTOP-TML9CJV:~$ echo 'Hello $USER !\n'
Hello $USER !\n
lihbyc@DESKTOP-TML9CJV:~$ echo "Hello $USER !\n"
-bash: !\n: event not found
lihbyc@DESKTOP-TML9CJV:~$ echo '$Hello $USER !\n'
Hello $USER !
lihbyc@DESKTOP-TML9CJV:~$
```

图 3: 三种引号输出对比

解题感悟： 单引号 (') 保留所有字面量，变量和转义序列不会被展开；双引号 (") 允许变量 (\$USER) 和命令替换，但转义序列如 \n 不会被解释；ANSI-C 引号 ('\$'...) 支持转义序列（如 \n 换行）。选择正确的引号可以避免脚本中的意外行为。

练习 4: 标准流重定向

练习描述： 本练习通过运行 ls 命令分别重定向标准输出和标准错误到不同文件，再合并重定向到同一文件，演示了 Shell 重定向机制。**命令：**

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
ls /nonexistent /tmp > both.txt 2>&1
```

```

lihbyc@DESKTOP-TML9CJV:~$ ls /nonexistent /tmp > stdout.txt 2> s
tderr.txt
lihbyc@DESKTOP-TML9CJV:~$ cat stdout.txt
/tmp:
hsperfdata_root
systemd-private-921060cf10344b2d9977b66f3e5d0564-systemd-logind.
service-FvgY4Y
lihbyc@DESKTOP-TML9CJV:~$ cat stderr.txt
ls: cannot access '/nonexistent': No such file or directory
lihbyc@DESKTOP-TML9CJV:~$ ls /nonexistent /tmp > both.txt 2>&1
lihbyc@DESKTOP-TML9CJV:~$ cat both.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
hsperfdata_root
systemd-private-921060cf10344b2d9977b66f3e5d0564-systemd-logind.
service-FvgY4Y

```

图 4: 重定向输出文件内容

解题感悟: 标准流分别是 stdin (0)、stdout (1)、stderr (2)。> 重定向 stdout, 2> 重定向 stderr。2>&1 将 stderr 合并到 stdout, 便于统一捕获。这为日志记录和错误处理提供了灵活手段。

练习 5: 条件创建目录

练习描述: 本练习利用 \$?、&& 和 || 实现条件逻辑, 仅在目录不存在时创建它。**命令:** [-d /tmp/mydir] || mkdir /tmp/mydir && echo " 创建成功或已存在"

```

lihbyc@DESKTOP-TML9CJV:~$ [ -d /tmp/mydir ] || mkdir /tmp/mydir
&& echo "创建成功或已存在"
创建成功或已存在
lihbyc@DESKTOP-TML9CJV:~$ ls -ld /tmp/mydir
drwxr-xr-x 2 lihbyc lihbyc 40 Aug 24 23:20 /tmp/mydir
lihbyc@DESKTOP-TML9CJV:~$ [ -d /tmp/mydir ] || mkdir /tmp/mydir
&& echo "创建成功或已存在"
创建成功或已存在
lihbyc@DESKTOP-TML9CJV:~$ ls -ld /tmp/mydir
drwxr-xr-x 2 lihbyc lihbyc 40 Aug 24 23:20 /tmp/mydir
lihbyc@DESKTOP-TML9CJV:~$ [ -d /tmp/mydir ] || mkdir /tmp/mydir
&& echo "创建成功或已存在"
创建成功或已存在

```

图 5: 两次执行的结果

解题感悟: \$? 保存上一条命令的退出状态 (0 成功, 非 0 失败)。&& 前一条成功时执行, || 前一条失败时执行。该命令利用 test -d 检查目录是否存在, 不存在则创建, 并输出提示。这种模式常用于脚本的初始化步骤。

练习 6: 文件存在性检查脚本

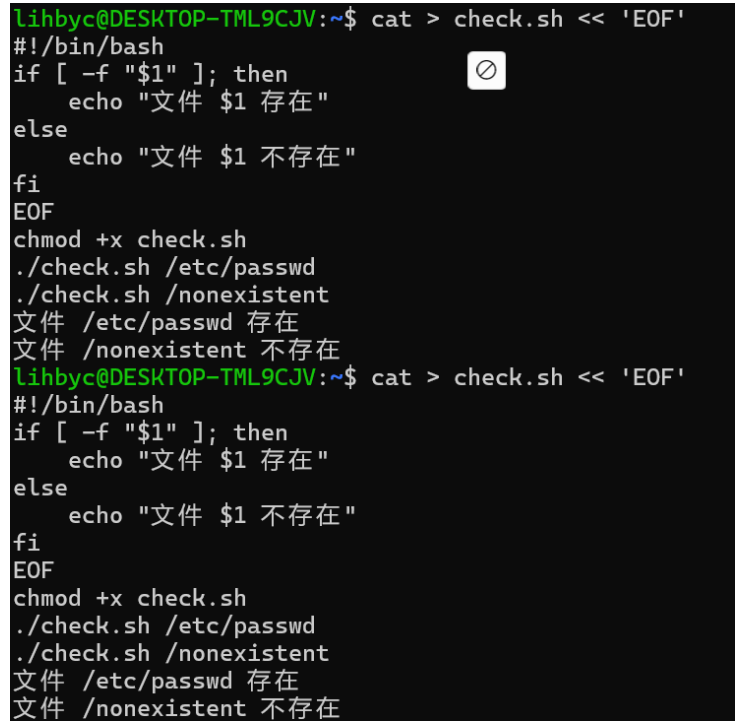
练习描述: 编写一个 Shell 脚本, 接收一个文件名参数, 使用 [-f ...] 判断该文件是否为普通文件并存在, 输出相应提示。**脚本内容:**

```

#!/bin/bash
if [ -f "$1" ]; then
    echo "文件 $1 存在"

```

```
else
    echo "文件 $1 不存在"
fi
```



```
lihbyc@DESKTOP-TML9CJV:~$ cat > check.sh << 'EOF'
#!/bin/bash
if [ -f "$1" ]; then
    echo "文件 $1 存在"
else
    echo "文件 $1 不存在"
fi
EOF
chmod +x check.sh
./check.sh /etc/passwd
./check.sh /nonexistent
文件 /etc/passwd 存在
文件 /nonexistent 不存在
lihbyc@DESKTOP-TML9CJV:~$ cat > check.sh << 'EOF'
#!/bin/bash
if [ -f "$1" ]; then
    echo "文件 $1 存在"
else
    echo "文件 $1 不存在"
fi
EOF
chmod +x check.sh
./check.sh /etc/passwd
./check.sh /nonexistent
文件 /etc/passwd 存在
文件 /nonexistent 不存在
```

图 6: check.sh 执行效果

解题感悟：使用 `[ -f ... ]` 测试普通文件是否存在，`$1` 接收第一个参数。脚本需要执行权限（`chmod +x`）才能直接运行，因为 Shell 不会执行没有执行权限的脚本文件。该脚本是文件前置检查的典型模板。

练习 7：使用 `set -x` 调试

练习描述：本练习通过 `set -x` 开启跟踪模式，观察脚本执行过程中每条命令的展开与执行情况。**脚本内容：**

```
#!/bin/bash
set -x
echo "Hello"
ls /tmp
set +x
echo "Done"
```

```
lihbyc@DESKTOP-TML9CJV:~$ cat > debug.sh << 'EOF'
#!/bin/bash
set -x
echo "Hello"
ls /tmp
set +x
echo "Done"
EOF
chmod +x debug.sh
./debug.sh
+ echo Hello
Hello
+ ls /tmp
hsperfdata_root
mydir
systemd-private-921060cf10344b2d9977b66f3e5d0564-systemd-logind.
service-FvgY4Y
+ set +x
Done
lihbyc@DESKTOP-TML9CJV:~$ |
```

图 7: set -x 输出的调试信息

解题感悟： set -x 开启跟踪模式，会打印每条命令及其参数（以 + 开头）；set +x 关闭跟踪。这是调试 Shell 脚本最常用的手段之一，可以快速定位错误所在行。

练习 8：备份文件带日期

练习描述： 本练习使用命令替换 \$(...) 动态生成带有当前日期的备份文件名。命令：

命令： cp notes.txt notes_\$(date +%Y-%m-%d).txt

```
lihbyc@DESKTOP-TML9CJV:~$ echo "test content" > notes.txt
cp notes.txt notes_$(date +%Y-%m-%d).txt
ls -l notes*.txt
-rw-r--r-- 1 lihbyc lihbyc 13 Aug 24 23:25 notes_2026-08-24.txt
-rw-r--r-- 1 lihbyc lihbyc 13 Aug 24 23:25 notes.txt
lihbyc@DESKTOP-TML9CJV:~$ |
```

图 8: 生成的带日期备份文件

解题感悟： 命令替换 \$(...) 执行内部命令并用其输出替换，此处 date +%Y-%m-%d 生成形如 2026-08-25 的日期。将备份文件名动态生成，避免了手动命名，适合定时任务脚本。

练习 9：find + xargs 统计行数

练习描述： 本练习使用 find 与 xargs 结合，查找所有 .sh 文件并统计行数。命令：

命令： find . -name "*.sh" -print0 | xargs -0 wc -l

```
lihbyc@DESKTOP-TML9CJV:~$ echo 'echo hello' > script1.sh
lihbyc@DESKTOP-TML9CJV:~$ echo 'echo world' > script2.sh
lihbyc@DESKTOP-TML9CJV:~$ echo 'echo world' > script2.sh
lihbyc@DESKTOP-TML9CJV:~$ find . -name "*.sh" -print0 | xargs -0
wc -l
19 ./q02/analyze.sh
1 ./script1.sh
6 ./check.sh
6 ./debug.sh
1 ./script2.sh
33 total
lihbyc@DESKTOP-TML9CJV:~$ |
```

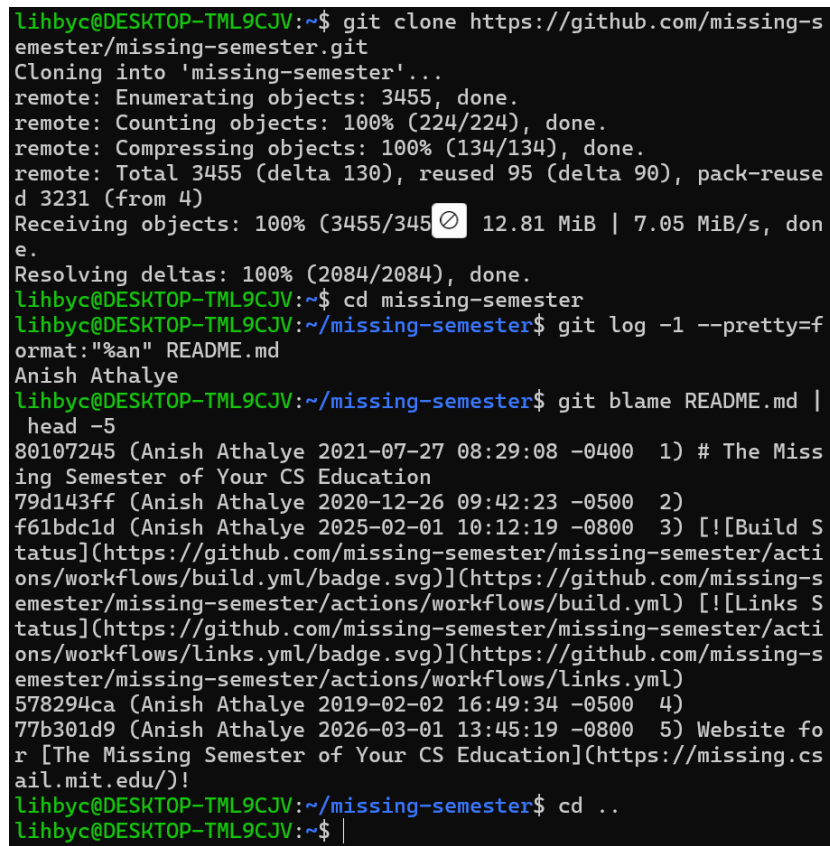
图 9: 统计各.sh 文件的行数

解题感悟： find 的 `-print0` 用空字符分隔文件名，`xargs -0` 相应地读取，这样能正确处理文件名中的空格和特殊字符。该组合比 `find -exec` 更高效，适合批量处理大量文件。

练习 10: Git 查看修改者信息

练习描述：本练习克隆一个真实仓库，使用 `git log` 和 `git blame` 查看文件的最后修改者和每行的修改细节。**命令：**

```
git clone https://github.com/missing-semester/missing-semester.git
cd missing-semester
git log -1 --pretty=format:"%an" README.md
git blame README.md | head -5
```



```
lihbyc@DESKTOP-TML9CJV:~$ git clone https://github.com/missing-semester/missing-semester.git
Cloning into 'missing-semester'...
remote: Enumerating objects: 3455, done.
remote: Counting objects: 100% (224/224), done.
remote: Compressing objects: 100% (134/134), done.
remote: Total 3455 (delta 130), reused 95 (delta 90), pack-reuse d 3231 (from 4)
Receiving objects: 100% (3455/3455) 12.81 MiB | 7.05 MiB/s, done.
Resolving deltas: 100% (2084/2084), done.
lihbyc@DESKTOP-TML9CJV:~$ cd missing-semester
lihbyc@DESKTOP-TML9CJV:~/missing-semester$ git log -1 --pretty=format:"%an" README.md
Anish Athalye
lihbyc@DESKTOP-TML9CJV:~/missing-semester$ git blame README.md | head -5
80107245 (Anish Athalye 2021-07-27 08:29:08 -0400 1) # The Missing Semester of Your CS Education
79d143ff (Anish Athalye 2020-12-26 09:42:23 -0500 2)
f61bdc1d (Anish Athalye 2025-02-01 10:12:19 -0800 3) [![[Build Status]](https://github.com/missing-semester/missing-semester/actions/workflows/build.yml/badge.svg)](https://github.com/missing-semester/missing-semester/actions/workflows/build.yml) [![[Links Status]](https://github.com/missing-semester/missing-semester/actions/workflows/links.yml/badge.svg)](https://github.com/missing-semester/missing-semester/actions/workflows/links.yml)
578294ca (Anish Athalye 2019-02-02 16:49:34 -0500 4)
77b301d9 (Anish Athalye 2026-03-01 13:45:19 -0800 5) Website for [The Missing Semester of Your CS Education](https://missing.csail.mit.edu/)!
lihbyc@DESKTOP-TML9CJV:~/missing-semester$ cd ..
lihbyc@DESKTOP-TML9CJV:~$
```

图 10: git log 和 git blame 输出

解题感悟：`git log -1 --pretty=format:"%an"` 显示最后一次修改 README.md 的作者姓名。`git blame` 逐行显示每行的最后修改提交信息（包括作者、提交哈希、时间等）。这些命令是代码审查和追溯责任的常用工具。

练习 11: 设置 Git 别名

练习描述：本练习通过 `git config` 创建一个自定义别名 `git graph`，将复杂的 `log` 命令简化为一个单词，提高日常工作效率。**命令：**

```
git config --global alias.graph "log --all --graph --decorate --oneline"
git graph
```

```
lihbyc@DESKTOP-TML9CJV:~$ git config --global alias.graph "log --all --graph --decorate --oneline"
lihbyc@DESKTOP-TML9CJV:~$ echo "test" > README.md
lihbyc@DESKTOP-TML9CJV:~$ git add README.md
lihbyc@DESKTOP-TML9CJV:~$ git commit -m "initial commit"
[master (root-commit) bd99cb4] initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
lihbyc@DESKTOP-TML9CJV:~$ git graph
* bd99cb4 (HEAD -> master) initial commit
lihbyc@DESKTOP-TML9CJV:~$ |
```

图 11: git graph 别名输出

解题感悟：Git 别名通过 `git config --global alias.<name> "<command>"` 设置，简化复杂命令。例如，`git graph` 替代了 `git log --all --graph --decorate --oneline`，提高了日常工作效率。

练习 12: 设置全局.gitignore

练习描述：本练习设置全局 Git 忽略文件，避免在每个仓库中重复配置忽略规则（如 macOS 的 `.DS_Store` 和日志文件）。**命令：**

```
git config --global core.excludesfile ~/.gitignore_global
echo ".DS_Store" >> ~/.gitignore_global
echo "*.log" >> ~/.gitignore_global
cat ~/.gitignore_global
```

```
lihbyc@DESKTOP-TML9CJV:~$ git config --global core.excludesfile
~/.gitignore_global
lihbyc@DESKTOP-TML9CJV:~$ echo ".DS_Store" >> ~/.gitignore_globa
l
lihbyc@DESKTOP-TML9CJV:~$ echo "*.log" >> ~/.gitignore_global
lihbyc@DESKTOP-TML9CJV:~$ cat ~/.gitignore_global
.DS_Store
*.log
lihbyc@DESKTOP-TML9CJV:~$ |
```

图 12: 全局忽略文件内容

解题感悟：全局 `.gitignore` 文件路径通过 `core.excludesfile` 指定，所有仓库都会忽略其中列出的文件模式（如 macOS 的 `.DS_Store` 和日志文件 `*.log`）。这避免了每个仓库重复配置，保持干净整洁。